Phase 5: RISC-V Multi Cycle Processor

EEL 4768 Computer Architecture: Spring 2026

Project Introduction

In this project phase, you will be building upon the previous phases of the project to build a multi cycle RISC-V 32RI processor in Verilog. A skeleton for each module will be provided, along with a top module used to interconnect each module, which you ***must use*** or the autograder may not be able to process your files.

Your processor should ***only use instructions found in the provided RV32I reference card posted in webcourses***. Using any instructions outside of the base RV32I instruction set will result in a zero for the specific problem.

Submission Requirements

You are permitted to use generative artificial intelligence (e.g. ChatGPT); if you choose to do so, indicate where you use AI in your submission, and why you used it. Please ensure your usage of LLMs adheres to the course policies.

Submit the Verilog files as individual, unzipped files directly to Gradescope. In addition to the Verilog files, please include a word document or pdf detailing the contributions of you and your team members, along with another document indicating your use of AI.

- RISCV_TOP.v
- Additional Verilog files
- assembler.(py, cpp)
- group_contributions.pdf
- ai_report.pdf
- signals.yaml

In Gradescope, after submitting you can select your group for that submission. One submission is needed, but if you fail to select your group members in that submission, they may receive a zero for this phase.

The points of this phase will be distributed as such,

5 points for a functioning multi-cycle processor.

1 point for each forwarding method.

1 point for submitting your group_contributions.py, for a total of 10 points.

Environment setup

You may use any simulator as long as it's capable of simulating Verilog. If you want to use Vivado similar to how you used it in digital systems, you are more than welcome to.

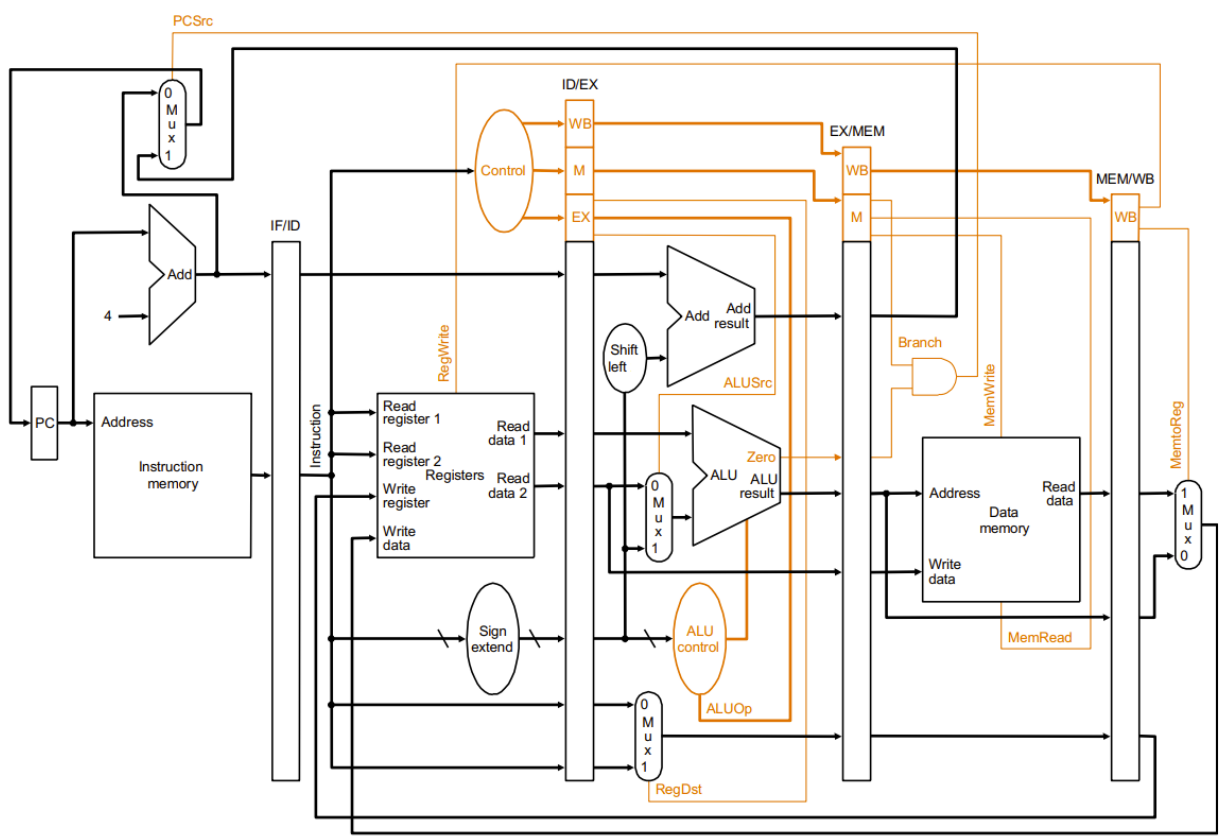
Another simulator that is recommended to be used if Verilator, which compiles the simulation of the Verilog files with a C++ wrapper. For assistance with installation, please come to my office hours.

Icarus is another lightweight option.

Your assembler can be writing in one of two languages of your choice.

- C++
- Python

Multi-Cycle Datapath



Here is an example of the multi cycle datapath for a RISC-V processor. Please use this as a reference to help visualize the datapath and signals. There may be discrepancies between the image above and the skeleton files, so make sure not to directly copy the details of this figure and only use it as a reference.

Your multi-cycle processor should handle data dependencies, such as

Read after write (RAW)

Read after read (RAR)

Write after read (WAR)

Write after write (WAW)

To handle each dependency, you need to implement the forwarding below.

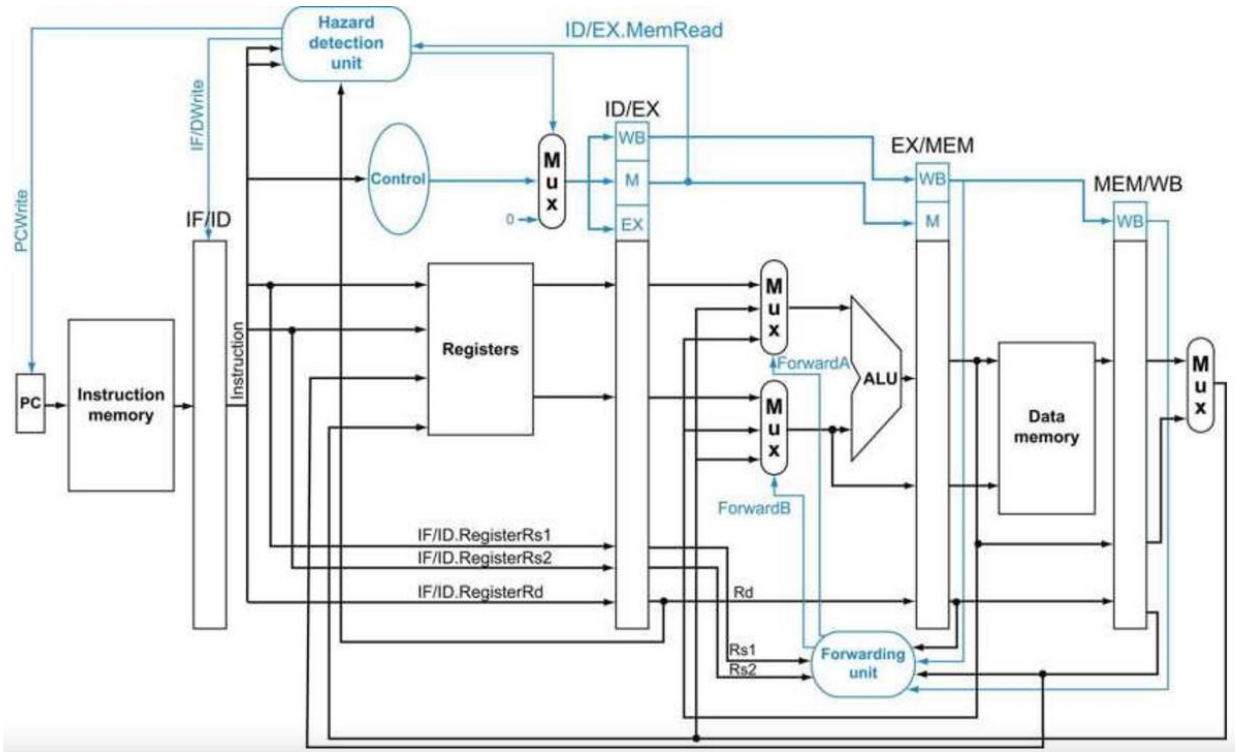
Ex – Ex forwarding

Mem – Ex forwarding

Mem – Mem forwarding

Load-to-use store stall

You will need additional hardware found in this multi-cycle figure, as shown below.



It will be up to you to make the additional modules necessary for the multi-cycle processor. Make sure your submission only includes Verilog files with one module per file. Each module should then be named the same as its file name.

For example, if your file was ALU.v, then it should have one module named ALU.

Skeleton

You may need to adjust the skeleton to function within a multi-cycle processor. This will be completely designed by you now.

Assembler

We will reuse our assemblers from Phase 2, but in this phase your assembler will be required to return two files of hex values, instr.txt and data.txt. Both should store a byte per line and use little endian. For example, if your instruction or data is

0x00AD7318

Then it should save as

0x18

0x73

0xAD

0x00

The instr.txt with contain only encoded instruction commands, while the data.txt will only contain word data that is initially stored in data.

The assembler should take in two arguments <file.s> and <output_directory>. The first argument should just be the input file, while the second argument is the output directory for your assembled instr.txt and data.txt files.

Your assembler should end each file with an ebreak signal, which is currently in each assembly file that is an input to your assembler. The ebreak instruction should decode to 0x00100073.

Branching and Jumping

Your processor should predict not taken when a branch or jump signal enters the pipeline and flush the processor when the branch is computed as taken. While sounding complex, this is the easiest way to implement control flow in the 5-stage pipeline. When a branch or jump instruction comes in, you do nothing. The PC continues to increment. When a branch or jump instruction is used to update the PC in the MEM stage, flush the register states so the invalid instructions don't save any data.

Signals.yaml

Finally, your submission should include a signals.yaml file, which is attached to this assignment. The contents will look like this.

```yaml
IF_ID_Instruction: ("wInstr") # pipelined register that holds the instrucion being output from the IF stage, not the PC register.
IF_ID_PC: ("wPC") # pipelined register that holds the PC being output from the IF stage.
Data_Memory: ("mem_stage", "data_memory", "rDataMem") # data memory registers
trace: [if_stage, id_stage, ex_stage, mem_stage, wb_stage, hd, fwd]
```

For those not familiar with yaml, it is a file that defines a data structure of key value pairs. A key, or name, will point to a value, which can be some object. In our example, they are string tuples and lists.

This file is needed by the internal simulator to find specific signals in your design. Selecting the wrong signals will make it not work.

IF_ID_Instruction:

This signal is the output register holding the instruction from your if/id stage.

IF_ID_PC:

This signal is the output register holding the PC from your if/id stage.

Data_Memory:

This signal points to your data memory register array.

<u>Making a signal:</u>

If the signal exists in your RISCV_TOP.v file, then all you need to do is put its name, as shown for the IF_ID_Instruction and IF_ID_PC. If a signal exists inside a module, then you need to add the module names before the name of the signal. These need to be the name of the instantiation, not the module itself.

Examples are shown on the next page, which align with my signals.

Finally, if you want additional tracing of your input, you can add the module instantiation names to the trace list. Every signal will be traced for you to view.

Example module structure:

```
module RISCV_TOP(
    …
);

Wire wInstr, wPC;

MEM mem_stage (…);
endmodule
```

```
module MEM(
    …
);

module DATA_MEMORY data_memory(…);
endmodule
```

```
module MEM(
    …
);

reg [B-1:0] rDataMem [0:(K*4)-1]; // 4KB data memory, byte-addressable
endmodule
```